



© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_1

1. What Are Browser Extensions?

Matt Frisbie¹

(1) California, CA, USA

Browser extensions are strange and powerful parasites. Browser extensions are parasites because they reside in an unusual corner of the world of software: they run on top of web pages yet are totally independent of any website or server. Browser extensions are powerful because they are afforded an unusual degree of agency: some browser extensions can view and modify everything you see in your browser, some can view every HTTP request your browser sends, and others can manage your digital currency wallets. Browser extensions are strange because they can directly antagonize the very companies that support them: uBlock Origin, one of the most popular Google Chrome extensions, exists only to prevent showing the very ads that make up Google's primary revenue channel.

Strange and powerful as they may be, browser extensions are an undoubtedly useful tool. They afford the end user a degree of control over their web browsing experience that is not possible in any other way. The most popular browser extensions have tens of millions of installs, and as of 2022, the Chrome Web Store has over 180,000 extensions published. In recognition of the broad adoption and utility, in 2021 the W3C formed the WebExtensions Community Group to “explore how browser vendors and other interested parties can work together to advance a common browser extension platform.”

Whatever lies ahead for browser extensions, it seems that they will not be going away any time soon.

History of Browser Extensions

Much in the same way that evolutionary biologists learn about modern life forms through the study of transitional fossils, browser extensions can be better understood by examining their software antecedents.

Customizing Software with Plugins

Support for *plugins*, software components that alter the behavior of an existing computer program, was first implemented in the 1970s on the UNIVAC Series 90 mainframe computer (Figure 1-1). EDT, a text editor running on the mainframe's Unisys VS/9 operating system, allowed other programs to “plug in” and access its in-memory buffer, as well as send commands back to it for processing. Granted these abilities, plugin programs were capable of programmatically altering the flow of other programs or invoking text editor commands, tasks that were normally accomplished via direct user input.



Figure 1-1 UNIVAC Series 90 mainframe computer

History of Web Browsers

The fundamentals of web browser behavior were a major contributor to the explosive adoption of the Internet. Unlike the conventional software delivery model of distributing executables – binary programs that are difficult to inspect or change – software delivered over HTTP was open source. Anyone who receives an HTML page has the ability to do what they like with it: view the source, modify existing content and styling, or duplicate it and reuse it for their own needs. Good ideas could quickly propagate because it was easy to reverse engineer how a website achieved a feature.

The release of JavaScript in 1995 continued with this idea. Because JavaScript is an interpreted language, scripts delivered to the browser via HTTP were open source. Furthermore, part of JavaScript's usefulness was the Document Object Model (DOM), which allowed scripts to use an API to programmatically inspect and modify what the browser was showing the user. Even decades later, it remains true that every web page in the world can be inspected and modified in JavaScript via the same set of browser APIs.

Native Browser Plugins

At the same time that JavaScript was getting on its feet, software companies were rolling out new ways that software could run in the browser. This software took the form of “browser plugins,” which were closed source modules that were able to run programs natively on the host computer from within the browser. Examples of these were Java applets, Adobe Flash, Microsoft Silverlight and ActiveX, and Apple QuickTime. JavaScript at the time was quite slow and feature-limited, so offloading computation to the host system allowed for browsers to utilize more system memory, persistent storage, and graphics APIs that would otherwise be off-limits. Because these plugins lived in a totally different execution context, they were largely sandboxed from the web page around them.

These plugins were adopted out of necessity, but they were replete with problems. Plugins were clunky to install and use, and because they were provided with privileged access to the host system, they introduced lots of browser bugs and security holes. As browsers became more and more capable with HTML5, CSS3, and more fully-featured ECMAScript specifications, these plugins served less of a purpose, and as a result modern browsers have dropped support for them almost entirely.

From Browser Add-ons to Extensions

As early as 1999, Internet Explorer and Firefox supported “add-ons” that were capable of customizing the browser, but these used proprietary integrations and therefore were siloed to a single browser and limited to that vendor’s add-on API. It wasn’t until September 2009 that the first modern browser extensions became available in Google Chrome. Unlike add-ons for other browsers, these Chrome extensions could be built using HTML, CSS, and JavaScript. They interacted with the browser using a JavaScript extension API, and developers could publish their extensions to the Chrome Web Store.

By June 2012, Chrome passed Internet Explorer as the most popular web browser, and the Chrome Web Store reached 750 million total installs. In the ensuing years, rival browsers began to progressively adopt the APIs and extension model that Google Chrome had pioneered.

The Browser Extension Landscape

In the wake of years of trial and error, the browser extensions of today are a mature software product. They enjoy broad adoption by developers and consumers, have well-defined APIs and documentation, and they can easily reach the end user via a robust app store pipeline. In this section, we’ll explore how to think about browser extensions,

where users go to install them, and what the different categories of browser extension are.

Comparing Mobile Apps and Browser Extensions

In many respects, modern browser extensions resemble mobile apps. They both explicitly declare which permissions they need from the host system, are restricted by well-defined security models, and utilize a broad collection of APIs to interact with that host. They are both bundled into releases and deployed via a vendor-specific app store. Both mobile app and browser extension releases undergo a review process, and updates are automatically downloaded and installed via the host system.

Whereas mobile apps typically exist as a standalone user interface, extensions usually are built to run atop one or many web pages with a supplementary user interface. Unlike mobile apps, browser extensions do not necessarily have a user interface; many popular browser extensions are nothing more than a piece of JavaScript that runs in the background to execute a handler upon some event.

Browser Extension Stores

All major browsers offer a store where extensions can be published and downloaded:

- Google Chrome extensions can be installed from the Chrome Web Store (<https://chrome.google.com/webstore>). This extension store is the largest and most popular.
- Safari extensions can only be downloaded from the Apple App Store.
- Mozilla Firefox extensions can be installed from the Add-ons marketplace (<https://addons.mozilla.org/>). Notably, Firefox is the only browser that allows for extensions to be used on both desktop and mobile devices.

- Microsoft Edge extensions can be installed from the Edge Add-ons marketplace (<https://microsoftedge.microsoft.com/addons/Microsoft-Edge-Extensions-Home>). The Edge browser's recent adoption of the Chromium rendering engine means that Chrome Web Store extensions can be installed in the Edge browser by enabling the "Allow extensions from other stores" setting.
- Opera extensions can be downloaded from the Opera Addons marketplace (<https://addons.opera.com/>). Like the Edge browser, Opera is a Chromium-based browser and can be configured to install extensions from the Chrome Web Store.

Types of Browser Extensions

Because of the broad API at their disposal, developers are capable of building browser extensions able to perform a remarkable range of tasks. However, just as Unix programs are expected to "do one thing and do it well," browser extensions are generally geared toward being smaller and more targeted in their purview.

Ad and Tracking Blockers

Easily the most common extension type, ad and tracking blockers serve a very simple purpose: to block unwanted content. Modern web pages are constructed from a cascade of requests for individual pieces of content. Most of these requests are for essential pieces of content like CSS and JavaScript to render the page, but many are for scripts that render ads, collect analytics, or track the user.

The core strategy these blocking extensions utilize is simple. When installed, a blocking extension is granted the ability to inspect and manage all HTTP requests a page makes. The extension also comes bundled with a list of URL domains and regular expressions that are known to serve ads or tracking content. As each page request goes out,

the extension checks it against the list, and kills the request if there is a match.

From the page's perspective, it will appear that the request simply failed – something that is completely normal and expected when sending requests to a remote resource. From the user's perspective, the web page will *usually* be correctly rendered with only the ads and tracking scripts teased out. From the ad or tracker server's perspective, they have no idea that the user ever visited the page, since the outgoing request never reaches the server.

Of course, this sort of extension is problematic for entities that depend on the blocked content to generate information or revenue. In response, the ad and tracker industries have begun to roll out countermeasures to address the widespread use of blocker extensions. Some pages now will only render their content if the ad and tracker scripts load; they assume that a failed request is *always* because a blocking extension killed the request. Some ad and tracking servers serve their scripts using “CNAME cloaking,” where the ad or tracking script request pretends to be regular content loaded from the page's domain. Furthermore, Google Chrome is using its dominant extension market share to champion a transition to a new manifest format, manifest v3, that will severely reduce the ability of extensions to effectively block ad and tracker requests.

NoteThe APIs used by extensions to manage network requests are covered more in the *Networking* chapter. The transition to manifest v3 is covered in depth in the *Extension Manifests* chapter.

Password Managers

Extensions can manage a JavaScript execution environment that is completely separated from any web page. This sandboxed environment is ideally suited to load and store secret information, where it can be securely passed to and from web pages on demand. Password

managers leverage this to record username/password combinations, save them locally or remotely, and type them in for the user.

A password manager extension allows the user to log in to the password manager service inside a sandboxed extension interface. Once credentialed, they will load the user's encrypted passwords from a remote server and decrypt them locally. With the passwords now available in memory, the extension manager is granted permissions that allow it to effectively manage a user's passwords:

- View and manage the page DOM to automatically detect username and password inputs, type in usernames and passwords, and render additional content on the page to manage user credentials
- View and manage outgoing requests to automatically detect requests that contain user credentials

Perhaps the most important aspect of using password managers is their inherent ability to almost completely prevent any phishing attempts. Password managers can accurately associate a user's credentials with the domains upon which they should be used. For example, a user's credentials on `realsite.com` should only ever be filled when the browser is on `realsite.com`. Suppose the user receives a phishing email and is tricked to click on a link that takes them to `fakesite.com` (pretending to be `realsite.com`). Even though `fakesite.com` may appear to the user to be an exact replica of `realsite.com`, the password manager extension sees the URL is not `realsite.com`, and it will decline to fill the credentials. Of course, this doesn't prevent the user from entering their password manually, but nevertheless the URL matching feature is a robust point of defense against this type of attack.

Smart Writing Management Tools

Extensions can be granted full access to the page DOM, and as a result they are capable of viewing and managing page content – including text inputs and their text content. Browser extensions that listen for

keyboard events and clicks are afforded a real-time feed of exactly what the user is typing. This text content can be piped through to any number of text management tools, such as spell checkers, grammar checkers, and smart writing assistants. These extensions can then style the text and insert popup boxes into the page that provide the user an on-demand interactive interface to manage the smart text management tools alongside the actual HTML input fields.

Accessibility Tools

The primary way to consume content on the Internet is via text, and not everyone has an easy time reading a computer screen. Just as smart writing management extensions can read and change input text, accessibility extensions can take the existing page text and format it in a way to make it more accessible to the user. This can take the form of automatically translating page text, piping the page text to screen reading software, reformatting the page text to make it more legible (by increasing size, contrast, or choosing alternate fonts), or by offering quick access to a dictionary.

Content and Link Aggregators

Plenty of web users find the need to quickly and easily save web page URLs or partial content of those pages. Aggregator extensions can add in a user interface to quickly grab URLs or portions of page HTML and save them, either locally or on a third-party server. Extensions also have access to the Bookmark API, allowing them to view and manage the browser's bookmarked sites.

Tab Management Tools

Browser extensions have access to an extensive tab management API, allowing them to perform a wide range of tab-centric actions that otherwise would need to be accomplished directly by the user. Custom new tab behavior, reordering existing tabs, discarding stale tabs, and

quickly accessing commonly used tabs are just a few of the possible features these types of extensions provide.

Screen Recording Tools

Not only are browser extensions capable of viewing the page DOM, they are also able to make use of HTML5 APIs to *literally* see what your browser page looks like. These are useful for tools like screen recording, screen sharing, and capturing screenshots.

Note Extensions that use these HTML5 APIs still require the user to explicitly give permission. Permission-gated APIs such as the Screen Capture API will still generate a browser dialog box asking the user to grant access when a browser extension requests it.

Integrations for Software Platforms

Plenty of software platforms with a public API have seen fit to publish browser extensions that can directly connect to that API. This is especially useful when the content sent to those APIs is drawn from a web page the user is currently viewing. Sending content to remote file storage, automated access to a user's calendar, and accessing a third-party note-taking API are just a few of the ways that browser extensions use tight integration with a web page to improve user experience.

Digital Currency Wallets

The increasingly popular world of digital currencies brings along with it a new way that users must manage payments. The details vary between platforms and currencies, but the tasks that digital currency users must perform generally fall into one of the following buckets:

- **The user must read a long string of characters presented to them.** This might be a cryptocurrency address that they wish to send funds to or receive funds from. These strings are usually

dozens of random characters, so typing or writing them down without making a mistake is impractical.

- **The user must perform a handshake with a platform.** This takes different forms depending on the technology involved, but digital currency platforms will often support some form of “connection” with your wallet that allows for payments to be sent or actions to be signed or explicitly authorized.

Browser extensions are well-suited for both tasks. Their ability to automatically read page content means long character strings become trivial to manage. Furthermore, their separate execution context is a very secure place for developers to host a digital wallet.

Developer Tools

Modern web development could not have happened without developer browser extensions. In the early days of the web, developers were desperate for ways to more easily debug the websites they were developing. One of the earliest solutions to this was a Firefox addon released in 2006 called Firebug, which allowed developers to view and tease apart the web page they were currently viewing (Figure 1-2). This greatly simplified their debug process for HTML, JavaScript, and CSS, which without the addon would incur a patchwork mess of using print statements and viewing the raw page source.

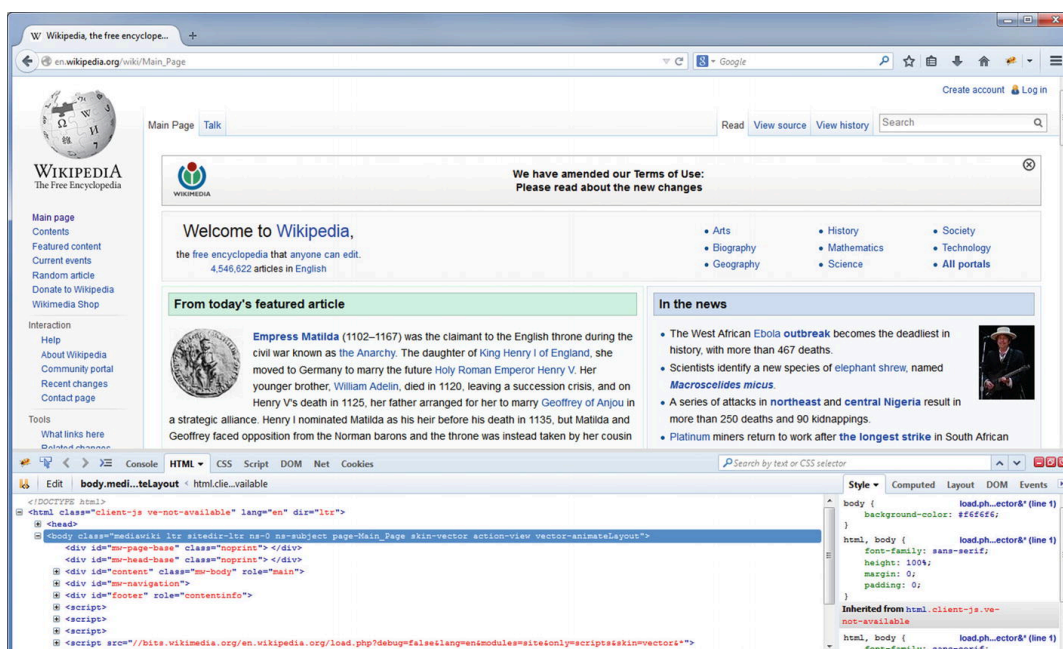


Figure 1-2 Firebug user interface

Browsers quickly caught on that there was incredible demand for this sort of tooling, and they began to roll out native versions of debuggers that matched the feature set of Firebug. Having become redundant, the Firebug extension was shut down in 2017.

With the mass adoption of single page applications like React, demand for developer tooling has followed in turn. The native browser debugger is not well-suited for debugging these sorts of applications, as the logic and architecture governing how the page renders is sequestered in large blobs of third-party JavaScript that is very difficult to debug.

To address this issue, many teams that work on these single page application frameworks also release a companion browser extension. The browser extension is intimately familiar with the internals of how a specific single page application behaves and is equipped with a broad toolkit that allows the developer to peer into the framework and understand what it is doing. This affords developers a much richer insight into what is happening on the page – and more importantly, how to fix it.

Furthermore, browser extensions have evolved to support direct integration into the browser's developer tools. Extensions can insert a custom *devtools page* into the browser's debug interface, interact with the inspected page, and debug network requests. They also have access to a custom DevTools API. The React Developer Tools user interface embedded inside the browser's developer tools is shown in Figure **1-3**.

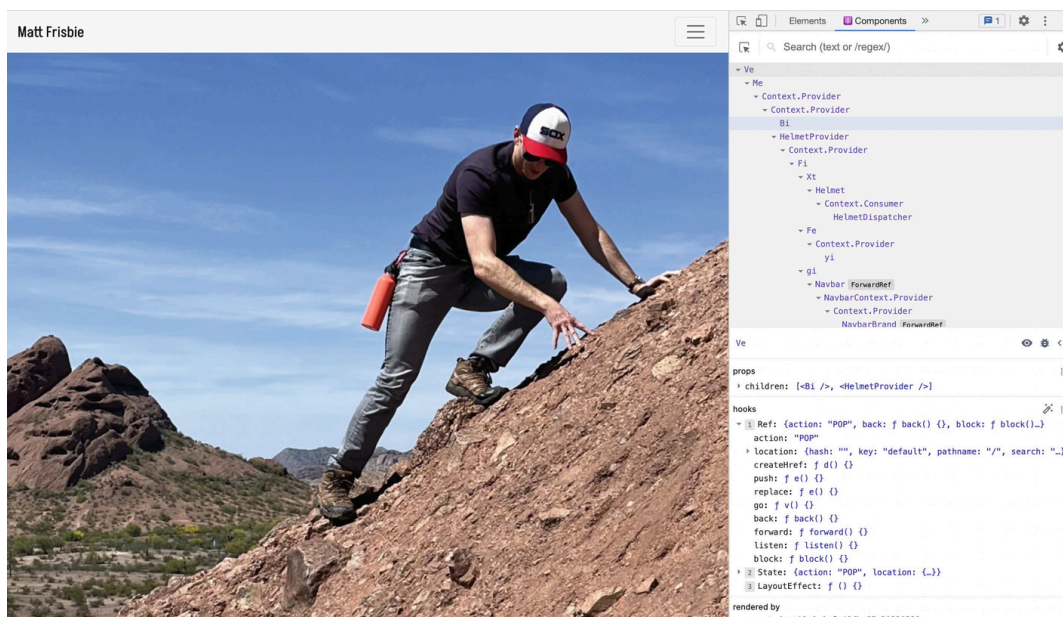


Figure 1-3 React Developer Tools user interface

Summary

Browser extensions have sneakily become an essential component of modern computing. From the humble of beginnings of the mainframe computer, they have evolved over decades into a veritable Swiss Army Knife for the modern browser.

The power of browser extensions is acutely expressed in the broad range of tooling that they enable. From powerful security and privacy tools like ad blockers and password managers, to a suite of tools for in-browser writing, to a broad set of single page application developer tools, browser extensions enable and augment more of the modern web experience than most people might realize.

The next chapter will explore all the different pieces that compose a browser extension.

Credits

Figure **1-1**:

<https://commons.wikimedia.org/wiki/File:Univac9060.jpg>

Source: United States Navy

Figure **1-2**:

https://commons.wikimedia.org/wiki/File:Firebug_extension_screenshot.png

Source: [Article: Wikipedia contributors Firebug: Firebug contributors screenshot: SteveSims, CC BY-SA 3.0](#), via Wikimedia Commons